Generative Models

Machine Learning (Bilingual)

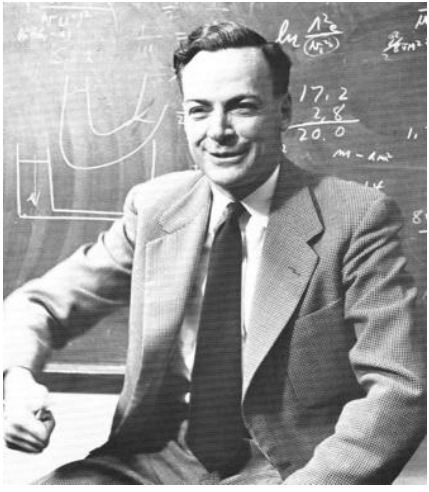
December 9, 2025

School of Artificial Intelligence
Xidian University

Contents

Introduction

Motivation: Why Generative Models?



Richard Feynman

What I cannot create,
I do not understand.

- **Goal:** Learn the true underlying distribution of data, $p_{data}(x)$, to generate new, realistic samples.
- **Key Applications:**
  - Image Synthesis (e.g., Faces, Bedrooms)
  - Text-to-Speech (e.g., WaveNet)
  - Super-resolution & Inpainting
  - DeepFakes & Content Creation

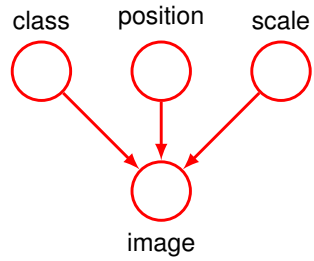
Generative vs. Discriminative Models

Discriminative Models

- Learn the **conditional** probability $P(Y|X)$.
- Focus: Finding the decision boundary between classes.
- *Example:* Classifying an image as a Cat or a Dog.

Generative Models

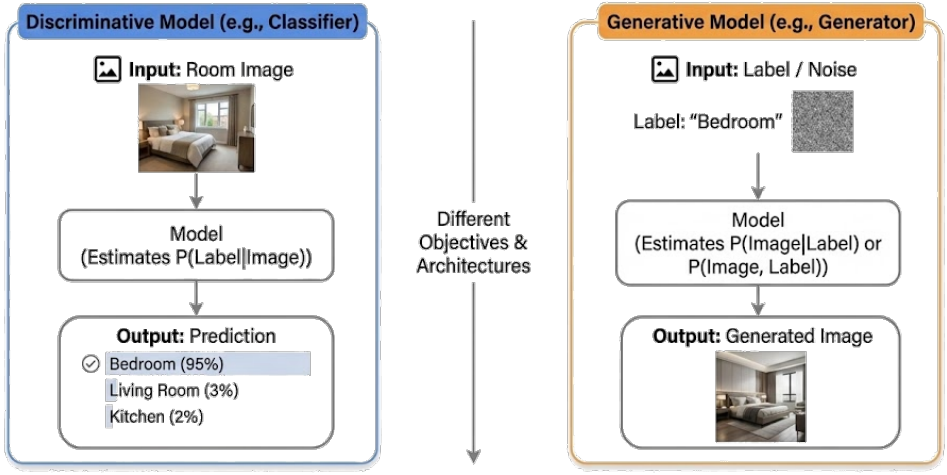
- Learn the **joint** probability $P(X, Y)$ or distribution $P(X)$.
- Focus: Understanding how the data is created.
- *Insight:* If you can generate the data, you understand its structure.



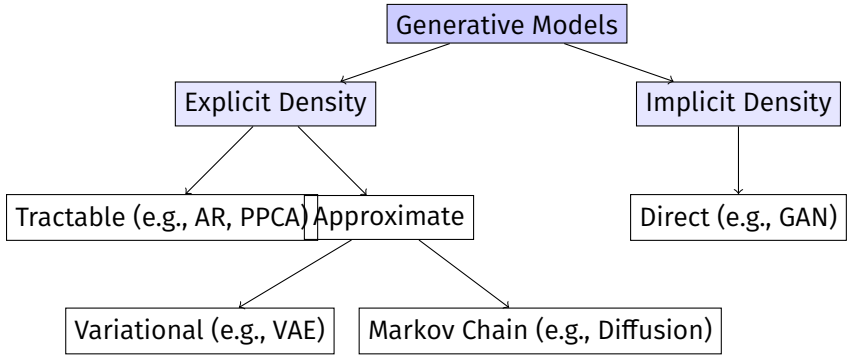
Quick Overview: Why Generative Models?

- **Discriminative:** model the conditional $P(Y \mid X)$ — optimized for prediction/classification.
- **Generative:** model the joint $P(X, Y)$ or data distribution $P(X)$ — enables **sampling**, density estimation, and data understanding.
- **Why we need them:**
  - **Sampling / Content Synthesis**: create realistic images, audio, text.
  - **Density Estimation / Anomaly Detection**: evaluate how likely a sample is.
  - **Imputation & Inverse Problems**: fill missing data, solve inverse tasks.
  - **Semi-/Self-supervised Learning**: leverage unlabeled data using $P(X)$.

Discriminative vs. Generative Models



Taxonomy of Generative Models



AR Autoregressive Models (e.g., PixelRNN, PixelCNN)
PPCA Probabilistic Principal Component Analysis
VAE Variational Autoencoders
GAN Generative Adversarial Networks

Likelihood-Based Models

Probabilistic PCA

Probabilistic PCA represents a constrained form of the Gaussian distribution in which the number of free parameters can be restricted while still allowing the model to capture the dominant correlations in a data set.

- Probabilistic PCA can be used to model class-conditional densities and hence be applied to classification problems.
- The probabilistic PCA model can be run generatively to provide samples from the distribution.
- Probabilistic PCA is a simple example of the **linear-Gaussian** framework, in which all of the marginal and conditional distributions are Gaussian.

Probabilistic PCA

- Let us first introducing an **explicit latent variable z** corresponding to the principal-component subspace.
- Next we define a Gaussian prior distribution $p(\mathbf{z})$ over the latent variable,

$$p(\mathbf{z}) = \mathcal{N}(\mathbf{z}|\mathbf{0}, \mathbf{I}),$$

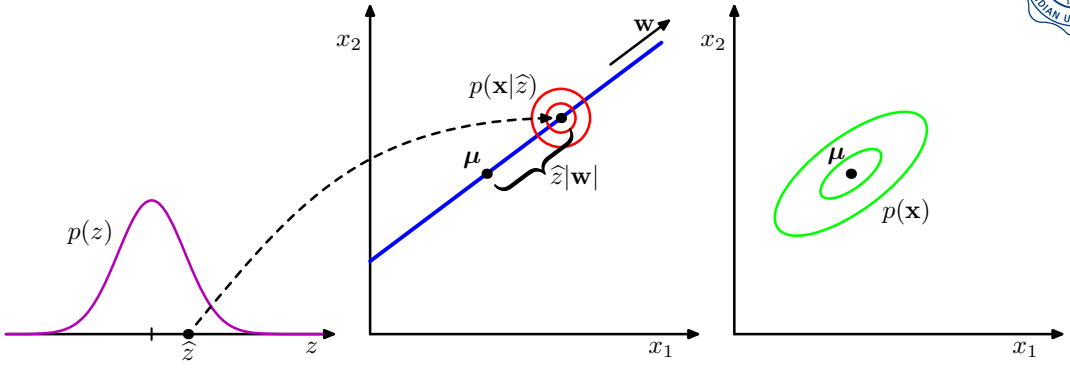
- together with a Gaussian conditional distribution $p(\mathbf{x}|\mathbf{z})$ for the observed variable x conditioned on the value of the latent variable.

$$p(\mathbf{x}|\mathbf{z}) = \mathcal{N}(\mathbf{x}|\mathbf{W}\mathbf{z} + \boldsymbol{\mu}, \sigma^2\mathbf{I})$$

- We can view the probabilistic PCA model from a generative viewpoint, so that

$$\mathbf{x} = \mathbf{W}\mathbf{z} + \boldsymbol{\mu} + \boldsymbol{\epsilon}.$$

Probabilistic PCA



Generative view of the probabilistic PCA model for a 2D data space and a 1D latent space. An observed data point **x** is generated by first drawing a value $\hat{z}$ for the latent variable from its prior distribution $p(\hat{z})$ and then drawing a value for **x** from an isotropic Gaussian distribution (illustrated by the red circles) having mean $\mathbf{w}\hat{z} + \boldsymbol{\mu}$ and covariance $\sigma^2\mathbf{I}$. The green ellipses show the density contours for the marginal distribution $p(\mathbf{x})$.

Probabilistic PCA

- Suppose we wish to determine the values of the parameters $\mathbf{W}$, $\boldsymbol{\mu}$ and σ^2 using maximum likelihood.
- We need an expression for the marginal distribution $p(\mathbf{x})$,

$$p(\mathbf{x}) = \int p(\mathbf{x}|\mathbf{z})p(\mathbf{z})\ \mathrm{d}\mathbf{z} = \mathcal{N}(\mathbf{x}|\boldsymbol{\mu}, \mathbf{C}),$$

where the $d \times d$ covariance matrix $\mathbf{C}$ is defined by

$$\mathbf{C} = \mathbf{W}\mathbf{W}^T + \sigma^2\mathbf{I}.$$

Probabilistic PCA

- When we evaluate the predictive distribution, we require $\mathbf{C}^{-1}$, which involves the inversion of a $d \times d$ matrix.
- Using the matrix inversion identity, we have

$$\mathbf{C}^{-1} = \sigma^{-2}\mathbf{I} - \sigma^{-2}\mathbf{W}\mathbf{M}^{-1}\mathbf{W}^T,$$

  where the $d' \times d'$ matrix $\mathbf{M}$ is defined by

$$\mathbf{M} = \mathbf{W}^T\mathbf{W} + \sigma^2\mathbf{I}.$$

- The posterior distribution $p(\mathbf{z}|\mathbf{x})$ is given by

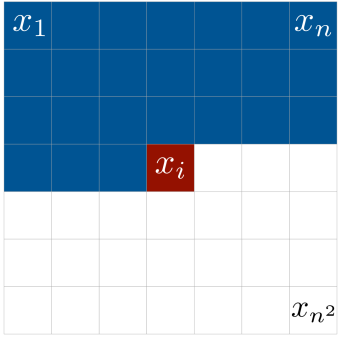
$$p(\mathbf{z}|\mathbf{x}) = \mathcal{N}\left(\mathbf{z}|\mathbf{M}^{-1}\mathbf{W}^T(\mathbf{x} - \boldsymbol{\mu}), \sigma^{-2}\mathbf{M}\right).$$

Autoregressive Models (AR)

Core Principles

Use the Chain Rule of Probability to decompose the joint distribution.

$$p(x) = \prod_{i=1}^{n} p(x_i | x_1, \ldots, x_{i-1})$$



Architectures:

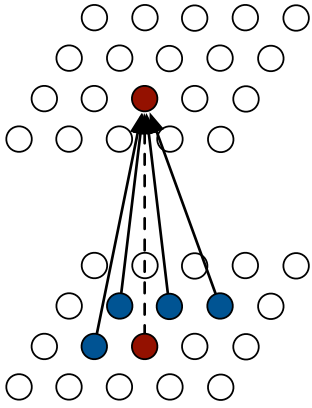
- **RNNs:** Sequential data (Text, Audio).
- **PixelRNN / PixelCNN:** Image generation pixel-by-pixel.

Pros & Cons

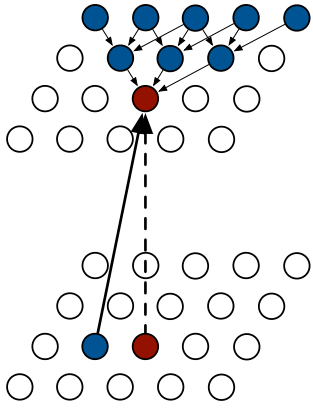
Pro: Exact likelihood calculation (Tractable). Easy to train.

Con: Sequential generation is computationally slow.

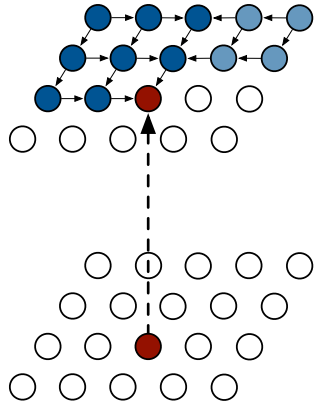
Autoregressive Models (AR): PixelRNN/PixelCNN



PixelCNN

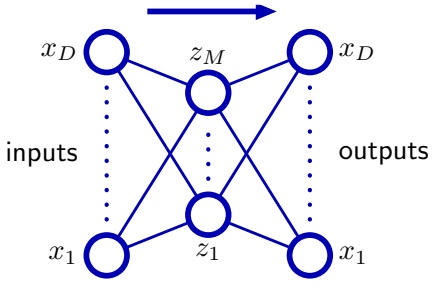


Row LSTM



Diagonal BiLSTM

Autoassociative Neural Networks[1]



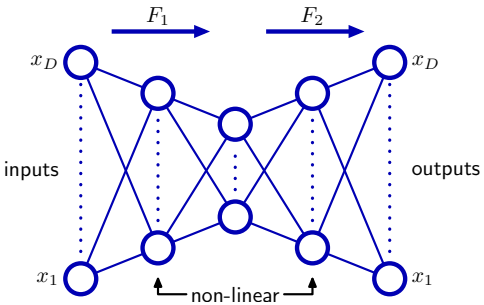
- Links representing bias parameters have been omitted for clarity.
- An autoassociative multilayer perceptron having two layers of weights.
- Such a network is trained to map input vectors onto themselves by minimization of a sum-of-squares error

$$E(\mathbf{w}) = \frac{1}{2}\sum_{n=1}^{N} \left\| y(\mathbf{x}_n, \mathbf{w}) - \mathbf{x}_n \right\|^2.$$

- Even with non-linear units in the hidden layer, such a network is equivalent to linear PCA.

[1] Hinton and Zemel, NIPS 1993.

Autoassociative Neural Networks (Autoencoder)



Addition of extra hidden layers of nonlinear units gives an autoassociative network which can perform a nonlinear dimensionality reduction.

Variational Autoencoders (VAEs)

- Introduce a latent variable z to capture factors of variation (pose, light, etc.).
- **Encoder** $q_\phi(z|x)$**:** Maps input to latent space.
- **Decoder** $p_\theta(x|z)$**:** Reconstructs input from latent.

The VAE Objective: The ELBO

We cannot optimize the intractable likelihood directly. Instead, we maximize the **Evidence Lower Bound (ELBO)**:

$$\log p(x) \geq \underbrace{\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)]}_{\text{Reconstruction Loss}} - \underbrace{D_{KL}(q_\phi(z|x)||p(z))}_{\text{Regularization Term}}$$

Trade-off

Pros: Principled approach, interpretable latent space.
Cons: Resulting images are often blurry (due to L2 reconstruction loss and Gaussian assumptions).

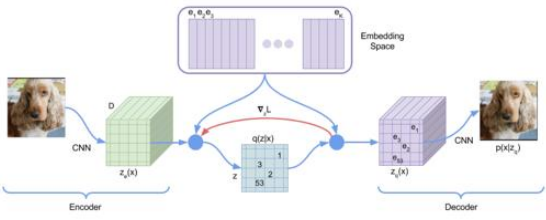
VQ-VAE: Discrete Latent Variables

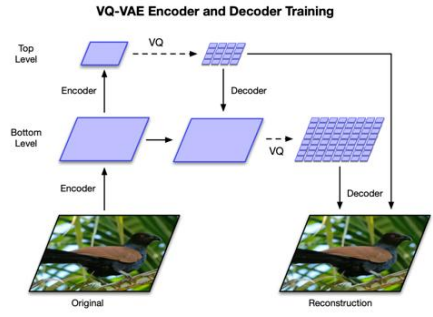
Motivation: Continuous latent spaces in VAEs can lead to blurry outputs.

VQ-VAE: Uses a discrete codebook of latent embeddings.

Encoder maps input to nearest codebook vector.

Decoder reconstructs input from selected codebook vector.





Implicit Generative Models

Generative Adversarial Networks (GANs)

- **Core Principle:** A Game Theoretic approach (Minimax Game).
- Two Neural Networks competing against each other:

Generator (G)

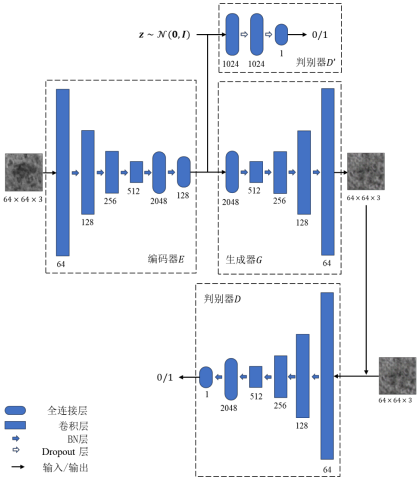
Tries to produce realistic fake data to fool the Discriminator.

$$z \rightarrow G(z)$$

Discriminator (D)

Tries to distinguish between Real data (x) and Fake data ($G(z)$).

The GAN Objective



The Generator and Discriminator play a Minimax game with the following Value Function $V(G, D)$:

$$\min_G \max_D V(G, D) = \mathbb{E}_{x \sim p_{data}}[\log D(x)] + \mathbb{E}_{z \sim p_z}[\log(1 - D(G(z)))]$$

- **Pros:** Generates very sharp, high-quality images.
- **Cons:** Training is unstable (Nash equilibrium is hard to find); Mode Collapse; No explicit likelihood density.

State-of-the-Art

Diffusion Models

The dominant paradigm in modern generative AI (e.g., Stable Diffusion, DALL-E).

- **Forward Process (Fixed):** Gradually add Gaussian noise to the data until it becomes pure noise (x_T).

$$x_0 \xrightarrow{\text{noise}} x_1 \dots \xrightarrow{\text{noise}} x_T$$

- **Reverse Process (Learned):** Learn a neural network to sequentially **denoise** the image.

$$x_T \xrightarrow{\text{denoise}} x_{T-1} \dots \xrightarrow{\text{denoise}} x_0$$

Advantage: High sample quality and training stability compared to GANs.

Summary

Summary Comparison

Model	Type	Pros	Cons
Autoregressive	Explicit (Tractable)	Exact Likelihood	Slow Sampling
VAE	Explicit (Approx)	Fast Sampling, Smooth Latent	Blurry Samples
GAN	Implicit	Sharp Samples	Unstable Training
Diffusion	Explicit (Approx)	High Quality, Stable	Slow Sampling (Iterative)